
PROBABILISTIC FORECASTING VIA AUTOREGRESSIVE FLOW MATCHING

Anonymous authors
Anonymous submission

ABSTRACT

We introduce FLOWTIME, a generative model for probabilistic forecasting in multivariate timeseries. Given historical observations and optional future covariates, the forecasting problem is specified as sampling from a learned conditional law over future trajectories. The proposed model factorizes the joint distribution of future observations into temporally ordered conditional densities; each density is represented by a shared flow that maps a simple base distribution to the next-observation distribution under the relevant covariate conditioning. These transformations are learned with the flow matching (FM) objective, which provides a scalable, simulation-free training criterion for conditional continuous flows. Combining the autoregressive factorization with FM preserves the operational advantages of autoregressive forecasting models, including extrapolation beyond the training horizon, compact parameterization, and calibrated predictive uncertainty. At the same time, the flow-based conditional density model can represent complex and multi-modal predictive distributions characteristic of modern transport-based generative models. We evaluate FLOWTIME on simulated dynamical systems and real-world forecasting benchmarks.

1 INTRODUCTION

Probabilistic timeseries forecasting is a central problem in modern machine learning: given observed measurements of a dynamical process, the objective is to characterize the conditional distribution of its future evolution. This formulation is required in scientific, industrial, and societal applications where downstream decisions depend not only on point predictions but also on calibrated uncertainty estimates (Lim & Zohren, 2021; Dama & Sinoquet, 2021; Ye et al., 2024).

Recent work increasingly addresses this conditional prediction problem with deep generative models (Karl et al., 2016; Rasul et al., 2020; Desai et al., 2021). Under this view, forecasting corresponds to drawing samples from a future conditional density parameterized by the observed history. Diffusion and score-based generative models (Sohl-Dickstein et al., 2015; Ho et al., 2020; Song et al., 2020) have produced strong results on several forecasting benchmarks (Rasul et al., 2021; Tashiro et al., 2021; Kollovich et al., 2023; Meijer & Chen, 2024). However, their training and inference procedures typically require many discretized denoising or score-integration steps, which increases computational cost.

Flow Matching (FM) (Liu et al., 2022; Albergo et al., 2023; Lipman et al., 2022) provides a general transport-based framework for generative modeling that includes diffusion-style constructions while allowing broader choices of probability paths. Rather than applying iterative stochastic denoising along a discretized trajectory, FM learns a deterministic vector field whose induced continuous normalizing flow transports a base distribution to the target distribution. Appropriate specification of the probability path removes the need for manually designed noise schedules and long ancestral sampling chains, thereby supporting more efficient training and generation.

FM has recently been applied to timeseries modeling and probabilistic forecasting (Tamir et al., 2024; Hu et al., 2024; Kollovich et al., 2024), where it has shown favorable empirical performance and improved computational efficiency relative to diffusion-based alternatives. Existing FM forecasting methods, however, commonly learn the conditional distribution of an entire fixed-length future window given a fixed-length context window. This direct window-level parameterization enables fast training and sampling, but it can degrade extrapolation outside the training distribution and

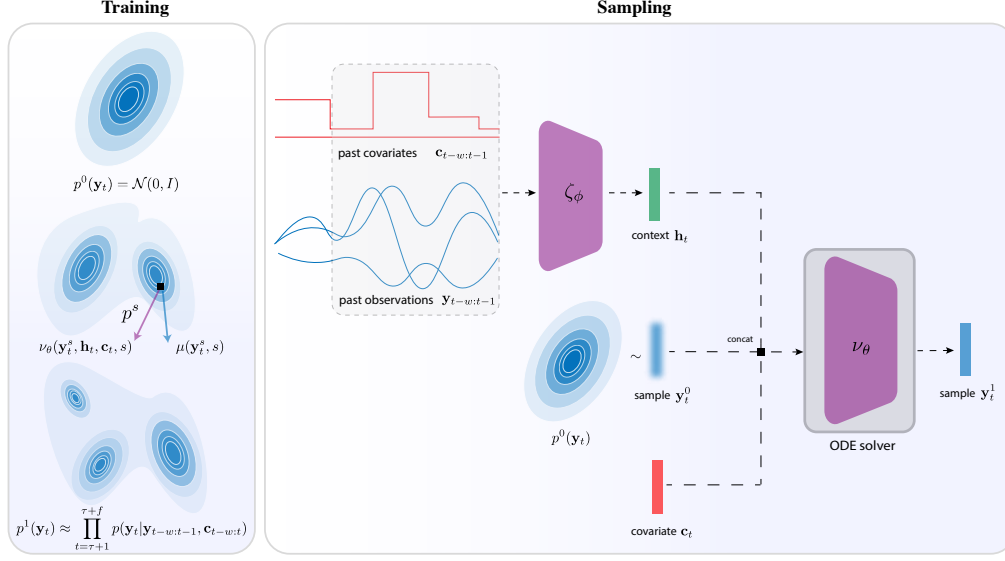


Figure 1: Training and sampling procedure for the proposed model. During training, a probability path $(p^s)_{0 \leq s \leq 1}$ is specified between a base distribution p^0 and the target distribution p^1 , and this path is induced by a velocity field μ . The model estimates μ with a neural network ν_θ whose inputs are a path sample $\mathbf{y}_t^s$ at flow step s , a context representation $\mathbf{h}_t$ summarizing past observations and covariates, and the contemporaneous covariate $\mathbf{c}_t$. At inference time, a sample is drawn from p^0 and transported to the target distribution by integrating the learned velocity field with an ODE solver until $s = 1$.

produce miscalibrated uncertainty estimates. It also yields a more difficult optimization problem, because the model must represent temporally structured probability paths along the flow dimension. This difficulty is noted by (Kollovich et al., 2024), who introduce Gaussian-process-informed priors to simplify and accelerate optimization; that design, however, restricts the method to univariate forecasting.

We propose FLOWTIME, an FM-based probabilistic forecasting model designed for high-dimensional multivariate data without the use of informed priors. Instead of modeling the future window as a single joint object, FLOWTIME decomposes the target conditional law into a sequence of one-step conditional distributions. Each conditional distribution represents the next time step given a finite window of previous observations and covariates, and all such conditionals share a flow that maps a simple base law to the corresponding next-observation law. This autoregressive construction accommodates variable forecast horizons, supports extrapolation beyond the training window, and provides a direct mechanism for propagating uncertainty through the forecast trajectory while reducing the dimensionality of each flow-matching subproblem.

Empirically, FLOWTIME obtains strong performance on classical stochastic dynamical systems and on real-world forecasting benchmarks, indicating that autoregressive flow matching is an effective formulation for probabilistic timeseries prediction.

2 BACKGROUND: FLOW MATCHING

Flow matching (FM) (Lipman et al., 2022) is a generative modeling framework for training continuous normalizing flows (CNFs) (Chen et al., 2018; Grathwohl et al., 2018) at large scale. In CNFs, data transformations are parameterized as solutions of ordinary differential equations (ODEs), yielding invertible maps with tractable likelihood evaluation.

As in CNFs, FM specifies a time-dependent diffeomorphic map on the data space Ω , denoted by the flow $\psi: [0, 1] \times \Omega \rightarrow \Omega$. The map transports a source sample $\mathcal{X}^0 \sim p$ to a terminal sample $\mathcal{X}^1 := \psi(\mathcal{X}^0, 1)$ whose distribution is the target law q . The flow is obtained as the solution of the

following initial-value problem:

$$d\psi(\mathcal{X}, s) = \mu(\psi(\mathcal{X}, s), s)ds, \quad \psi(\mathcal{X}, 0) = \mathcal{X}, \quad s \in [0, 1] \quad (1)$$

where $\mu: [0, 1] \times \Omega \rightarrow \Omega$ denotes the velocity field of the flow. This vector field induces a probability path $(p^s)_{0 \leq s \leq 1}$ over Ω , initialized at $p^0 = p$. The learning objective is therefore to recover a vector field μ whose terminal marginal satisfies $p^1 = q$.

Standard CNF training estimates the vector field by likelihood maximization, which requires solving and differentiating through Eq. (1) during optimization. FM instead parameterizes a vector field ν_θ with parameters θ and trains it by regression against vector fields associated with fixed conditional probability paths. This construction yields the tractable conditional flow matching (CFM) objective:

$$\mathcal{L}_{\text{CFM}}(\theta) = \mathbb{E}_{s \sim \mathcal{U}[0,1], \mathcal{X}^1 \sim q(\mathcal{X}^1), \mathcal{X} \sim p^s(\mathcal{X}|\mathcal{X}^1)} \|\nu_\theta(\mathcal{X}, s) - \mu(\mathcal{X}, s|\mathcal{X}^1)\|^2. \quad (2)$$

After minimizing this objective, samples from q are generated by solving Eq. (1) with the learned field ν_θ . Because the training loss does not require backpropagation through the ODE solver, FM supports stable and scalable optimization in high-dimensional generative domains, including image and video modeling (Esser et al., 2024; Polyak et al., 2024). Different probability-path specifications and conditioning schemes instantiate a broad class of transport-based generative models within this framework (Liu et al., 2022; Albergo et al., 2023; Tong et al., 2023).

3 AUTOREGRESSIVE FLOW MATCHING

Problem Setting Let $\mathbf{y}_\tau \in \mathbb{R}^n$ denote an n -dimensional observation at time τ . Given the l most recent measurements, including the current observation, $\mathbf{Y}_l = \{\mathbf{y}_{\tau-l}, \dots, \mathbf{y}_\tau\}$, the forecasting target is the distribution of the next f observations $\mathbf{Y}_f = \{\mathbf{y}_{\tau+1}, \dots, \mathbf{y}_{\tau+f}\}$. The model may additionally condition on observed covariates $\mathbf{C} = \{\mathbf{c}_{\tau-l}, \dots, \mathbf{c}_{\tau+f}\}$ with $\mathbf{c}_\tau \in \mathbb{R}^c$. Formally, given a dataset $\mathcal{D} = \{(\mathbf{Y}_f^i, \mathbf{Y}_l^i, \mathbf{C}^i)\}_{i=1}^m$ of m instances, the objective is to learn a sampler for $p(\mathbf{Y}_f|\mathbf{Y}_l, \mathbf{C})$.

We impose an order- w Markov structure over future time steps and factorize the conditional distribution autoregressively as:

$$p(\mathbf{Y}_f|\mathbf{Y}_l, \mathbf{C}) = \prod_{t=\tau+1}^{\tau+f} p(\mathbf{y}_t|\mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t}) \quad (3)$$

where $w \leq l$ is the history size. The learning problem is therefore reduced to estimating the one-step conditional factors in Eq. (3). Rather than fitting these factors by maximum likelihood, as is common in autoregressive density models, we estimate them with the flow matching framework (Lipman et al., 2022).

This factorization has several technical consequences. First, it replaces direct estimation of a high-dimensional future-window law with a family of lower-dimensional conditional distributions $p(\mathbf{y}_t|\mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t})$. Second, the conditional for each time step can be optimized under teacher forcing, which permits parallel construction of training examples. Third, the window size w explicitly controls the trade-off between representational capacity and computational cost. Finally, uncertainty propagation is represented through the product of conditional laws, making the temporal accumulation of predictive uncertainty explicit.

3.1 CONSTRUCTING A TARGET PROBABILITY PATH

For each future time step $t \in [\tau+1, \tau+f]$, we specify a probability path $(p^s(\mathbf{y}_t))_{0 \leq s \leq 1}$ with endpoint marginals $p^0(\mathbf{y}_t) = \mathcal{N}(0, I)$ and $p^1(\mathbf{y}_t) \approx \prod_{t=\tau+1}^{\tau+f} p(\mathbf{y}_t|\mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t}) \approx p_{\mathcal{D}}$, where $p_{\mathcal{D}}$ denotes the empirical distribution. The path is induced by a flow field $\psi: [0, 1] \times \mathbb{R}^n \rightarrow \mathbb{R}^n$ along the flow coordinate s , defined by the following ODE:

$$d\psi(\mathbf{y}_t, s) = \mu(\psi(\mathbf{y}_t, s), s)ds, \quad \psi(\mathbf{y}_t, 0) = \mathbf{y}_t \quad s \in [0, 1] \quad (4)$$

where $\mu: [0, 1] \times \mathbb{R}^n \rightarrow \mathbb{R}^n$ is the velocity field used to construct a flow whose induced probability path satisfies the continuity equation.

To obtain a tractable regression objective, the marginal probability path is represented as a mixture of conditional probability paths:

$$p^s(\mathbf{y}_t) = \int p^s(\mathbf{y}_t|\mathbf{z})p(\mathbf{z})d\mathbf{z} \quad (5)$$

where $\mathbf{z} \sim \pi_{0,1}(\mathbf{z})$ is a conditioning variable drawn from an arbitrary coupling $\pi_{0,1}$ between the end-point distributions. Theorem 2 of (Lipman et al., 2022) establishes that regressing on the conditional velocity fields $\mu(\mathbf{y}_t, s|\mathbf{z})$ associated with $p^s(\mathbf{y}_t|\mathbf{z})$ recovers the corresponding marginal velocity field $\mu(\mathbf{y}_t, s)$.

Following (Tong et al., 2023), we set $\mathbf{z} = \{\mathbf{y}_t^0, \mathbf{y}_t^1\}$ with $p(\mathbf{z} = (\mathbf{y}_t^0, \mathbf{y}_t^1)) = p^0 \times p^1$, where $\mathbf{y}_t^0$ and $\mathbf{y}_t^1$ are endpoint samples from p^0 and p^1 , respectively. The conditional probability path is specified as:

$$p^s(\mathbf{y}_t|\mathbf{z}) = \mathcal{N}((1-s)\mathbf{y}_t^0 + s\mathbf{y}_t^1, \sigma^2 I) \quad (6)$$

with endpoint marginals satisfying $p^0(\mathbf{y}_t|\mathbf{z}) = p^0(\mathbf{y}_t)$ and $p^1(\mathbf{y}_t|\mathbf{z}) = p^1(\mathbf{y}_t)$ in the limit $\sigma^2 \rightarrow 0$. The associated conditional velocity field is $\mu(\mathbf{y}_t, s|\mathbf{z}) = \mathbf{y}_t^1 - \mathbf{y}_t^0$. This construction defines straight Euclidean probability paths between source and target samples and admits closed-form training targets. Once this velocity field is learned, samples from $p(\mathbf{y}_t|\mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t})$ are obtained by solving the ODE in Eq. (4) to terminal flow time $s = 1$.

3.2 TRAINING

As in standard FM, the velocity field is estimated by regressing a neural network ν_θ with parameters θ against the conditional flow matching target in Eq. 2. For the forecasting setting, we introduce a context vector $\mathbf{h}_t \in \mathbb{R}^h$ that encodes the time- t context window:

$$\mathbf{h}_t = \zeta_\phi(\mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t-1}) \quad (7)$$

where $\zeta: \mathbb{R}^{w \times n} \times \mathbb{R}^{w \times c} \rightarrow \mathbb{R}^h$ is a neural encoder with parameters ϕ . The resulting learning objective is:

$$\mathcal{L}(\theta, \phi) = \mathbb{E}_{\mathbf{z} \sim \pi_{0,1}, s \sim \mathcal{U}(0,1), \mathbf{y}_t^s \sim p^s(\mathbf{y}_t|\mathbf{z})} \|\mu(\mathbf{y}_t^s, s|\mathbf{z}) - \nu_\theta(\mathbf{y}_t^s, \mathbf{h}_t, \mathbf{c}_t, s)\|^2 \quad (8)$$

The objective is optimized by sampling a time index t , the corresponding observation, its context window, and covariates from $\mathcal{D}$, and then jointly updating ϕ and θ to minimize $\mathcal{L}(\theta, \phi)$ with stochastic gradient descent. This training protocol is equivalent to teacher forcing (Williams & Zipser, 1989): ground-truth past observations, rather than generated past predictions, are supplied to the model during optimization. Consequently, training examples for different time steps can be processed in parallel and do not accumulate rollout error. Algorithm 1 gives the resulting autoregressive flow-matching training procedure.

Algorithm 1 Training: Autoregressive Flow Matching.

Require: Dataset $\mathcal{D} = \{(\mathbf{Y}_f^i, \mathbf{Y}_t^i, \mathbf{C}^i)\}_{i=1}^m$, window size w , networks ν_θ, ζ_ϕ

- 1: **while** not converged **do**
 - 2: Sample $\mathbf{y}_t^0 \sim \mathcal{N}(0, I)$ and $(\mathbf{y}_t^1, \mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t}) \sim \mathcal{D}$
 - 3: Set $\mathbf{h}_t = \zeta_\phi(\mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t-1})$
 - 4: Sample $s \sim \mathcal{U}(0, 1)$ and $\mathbf{y}_t^s \sim \mathcal{N}((1-s)\mathbf{y}_t^0 + s\mathbf{y}_t^1, \sigma^2 I)$
 - 5: Compute target: $\mu(\mathbf{y}_t^s, s|\mathbf{z}) = \mathbf{y}_t^1 - \mathbf{y}_t^0$
 - 6: Compute loss: $\mathcal{L}_t = \|\mu(\mathbf{y}_t^s, s|\mathbf{z}) - \nu_\theta(\mathbf{y}_t^s, \mathbf{h}_t, \mathbf{c}_t, s)\|^2$
 - 7: Update θ, ϕ
 - 8: **end while**
 - 9: **return** ν_θ, ζ_ϕ
-

3.3 INFERENCE

At inference time, the trained model defines a predictive sampler for a new history $\mathbf{Y}_t$ and associated covariates $\mathbf{C}$ over the next f time points. Algorithm 2 implements this sampler autoregressively with a rolling context window: after each one-step sample is generated, it is appended to the history used for subsequent conditional distributions. Repeating this sampling procedure yields Monte Carlo samples from which empirical predictive quantiles and uncertainty intervals are computed.

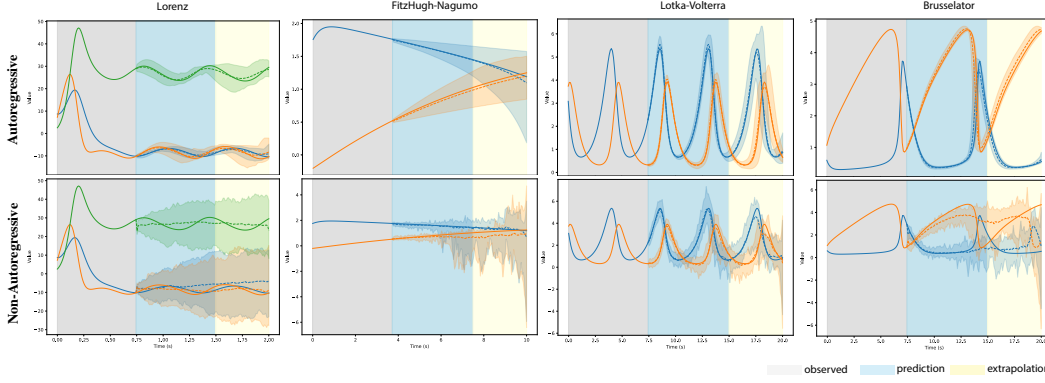


Figure 2: Representative forecast samples for autoregressive flow matching and standard flow matching on four stochastic dynamical systems. Solid curves denote ground-truth trajectories; dashed curves denote predictive means, with shaded 95% confidence intervals. Each panel reports behavior in both the prediction window and the extrapolation window.

Algorithm 2 Sampling: Autoregressive Flow Matching.

Require: Past observations $\mathbf{Y}_l$, covariates $\mathbf{C}$, forecast horizon f , window size w , networks ν_θ, ζ_ϕ

- 1: Initialize $\mathbf{Y}_f \leftarrow \mathbf{Y}_l$
 - 2: **for** $t = \tau + 1$ **to** $\tau + f$ **do**
 - 3: Set context: $\mathbf{h}_t = \zeta_\phi(\mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t-1})$, using the last w entries in $\mathbf{Y}_f$
 - 4: Sample $\mathbf{y}_t^0 \sim \mathcal{N}(0, I)$
 - 5: Solve $\frac{d\psi(\mathbf{y}_t, s)}{ds} = \nu_\theta(\psi(\mathbf{y}_t, s), \mathbf{h}_t, \mathbf{c}_t, s), \quad \psi(\mathbf{y}_t, 0) = \mathbf{y}_t^0, \quad s \in [0, 1]$
 - 6: Append $\psi(\mathbf{y}_t, 1)$ to $\mathbf{Y}_f$
 - 7: **end for**
 - 8: **return** $\mathbf{Y}_f$
-

4 EXPERIMENTS

This section evaluates FLOWTIME on two classes of forecasting problems: synthetic trajectories generated from stochastic dynamical systems and real-world timeseries benchmarks. Across both settings, performance is compared against baselines that isolate the effect of the autoregressive flow-matching formulation.

4.1 DYNAMICAL SYSTEMS

Data We evaluate FLOWTIME on forecasting tasks derived from five stochastic dynamical systems: stochastic versions of the Lorenz system (Lorenz, 1963), FitzHugh–Nagumo model (FitzHugh, 1961), Lotka–Volterra system Lotka (1925), Brusselator (Lefever & Prigogine, 1968), and Van der Pol oscillator (van der Pol, 1926). For each system, trajectories are generated by numerical integration of the corresponding stochastic differential equation under independently sampled initial conditions and Brownian-motion realizations. Appendix A.1 specifies the system equations and data-generation protocol.

Implementation details In these experiments, ζ_ϕ is implemented as a two-layer bidirectional LSTM with 64 hidden units, and ν_θ is implemented as an MLP with three hidden layers of width 64. The flow coordinate s is encoded with a fixed-frequency Fourier positional embedding of dimension 16, and the context length w is set equal to the prediction length. Models are optimized with Adam using learning rate 0.003 and batch size 128. Training to convergence required approximately ~ 7 minutes on one A100-SXM4-40GB NVIDIA GPU.

Baselines To isolate the contribution of the autoregressive factorization, we compare FLOWTIME with a non-autoregressive flow-matching baseline that directly models the full conditional future-window distribution, i.e. $p^1 = p(\mathbf{Y}_f | \mathbf{Y}_l, \mathbf{C})$. This baseline constitutes a direct ablation of

the temporal factorization used by FLOWTIME. Its formulation is closely related to the method of (Kollovich et al., 2024); implementation details are given in Appendix B.

Evaluation To evaluate extrapolation beyond the training distribution, each trajectory is partitioned into three contiguous segments: (1) an observation window used as historical input, (2) a prediction window used as the supervised target during training, and (3) an extrapolation window that is not observed during training. Predictive distribution quality is measured with the continuous ranked probability score (CRPS) (Winkler et al., 1996), and point-prediction accuracy is measured with normalized root mean square error (NRMSE) (Hyndman & Koehler, 2006).

Table 1: Test-set comparison between the autoregressive factorization (FLOWTIME) and the non-autoregressive factorization (Non-AR) on five stochastic dynamical systems. CRPS and NRMSE are reported separately for the prediction and extrapolation regimes. Values denote the mean and standard deviation over five random seeds.

System	Factorization	Prediction		Extrapolation	
		CRPS ↓	NRMSE ↓	CRPS ↓	NRMSE ↓
Lorenz	Non-AR	0.147±0.012	0.242±0.018	0.120±0.009	0.301±0.024
	FLOWTIME	0.090±0.008	0.017±0.003	0.137±0.011	0.048±0.006
FitzHugh-Nagumo	Non-AR	0.165±0.014	0.131±0.011	0.214±0.016	0.306±0.022
	FLOWTIME	0.144±0.013	0.090±0.007	0.146±0.012	0.278±0.019
Lotka-Volterra	Non-AR	0.130±0.011	0.266±0.019	0.136±0.012	0.502±0.033
	FLOWTIME	0.126±0.010	0.189±0.015	0.123±0.009	0.339±0.026
Brusselator	Non-AR	0.217±0.016	0.658±0.042	0.238±0.019	1.046±0.057
	FLOWTIME	0.125±0.011	0.031±0.004	0.109±0.008	0.023±0.003
Van der Pol	Non-AR	0.143±0.012	0.161±0.014	0.211±0.018	0.510±0.031
	FLOWTIME	0.225±0.017	0.051±0.005	0.282±0.021	0.101±0.009

Results Table 1 shows that the autoregressive factorization used in FLOWTIME improves forecasting performance on the stochastic dynamical-system benchmarks. In the prediction regime, the autoregressive model attains lower NRMSE than the non-autoregressive baseline on all five systems, with the largest reductions on Lorenz (0.017 versus 0.242) and Brusselator (0.031 versus 0.658), corresponding to error reductions of 93% and 95%. The autoregressive factorization also improves CRPS on four of the five systems; the only exception is Van der Pol, where the non-autoregressive model obtains lower CRPS. The same pattern is stronger in extrapolation: the autoregressive model has lower NRMSE on every system, including a 98% reduction for Brusselator (0.023 versus 1.046). These results indicate that one-step conditional flow matching provides a better inductive bias for representing the local transition structure of these stochastic systems, which in turn improves predictive accuracy and, in most cases, uncertainty calibration under both interpolation and extrapolation regimes.

4.2 REAL-WORLD DATASETS

Data We evaluate FLOWTIME on univariate and multivariate forecasting tasks from GluonTS (Alexandrov et al., 2020), covering both hourly and daily sampling frequencies. The benchmark set consists of Electricity (Lai et al., 2018), Solar (Lai et al., 2018), Exchange (Dua et al., 2017), Traffic (Dua et al., 2017), and Wikipedia (Gasthaus et al., 2019). Dataset properties are reported in Appendix A.2.

Implementation details For the real-world benchmarks, ζ_ϕ is a three-layer bidirectional LSTM with 64 hidden units. The velocity network ν_θ is a residual neural network with five residual blocks, implemented with 1-D convolutional layers and gated activation functions (Van Den Oord et al., 2016; Kong et al., 2020). This architecture follows the network design used in (Rasul et al., 2021), but uses fewer residual blocks. The flow step s is encoded as a 32-dimensional Fourier positional

embedding with fixed frequencies, and the same embedding construction is used for time-dependent covariates $\mathbf{c}$. Models are optimized with Adam using learning rate 0.001 and batch size 128. Depending on the dataset, convergence required 40 – 100 minutes on one A100-SXM4-40GB NVIDIA GPU.

Baselines We compare FLOWTIME with established statistical, autoregressive neural, diffusion-based, and flow-matching forecasting methods. The statistical baselines are Seasonal Naive (SN), AutoARIMA, and AutoETS (Hyndman et al., 2008). Neural baselines include DeepAR (Salinas et al., 2020) and WaveNet (Van Den Oord et al., 2016). Generative baselines include the diffusion models TSDiff (Kollovieh et al., 2023), SSSD (Alcaraz & Strodtzoff, 2022), and TimeGrad (Rasul et al., 2021), as well as the flow-matching method TSFlow (Kollovieh et al., 2024).

Table 2: Test-set CRPS comparison for five real-world forecasting datasets, where lower values indicate better predictive distributions. FLOWTIME entries report the mean and standard error over five independent training and evaluation runs. Baseline results are taken from (Kollovieh et al., 2024). Best scores are shown in **bold**; second-best scores are underlined.

Method	Electr.	Exchange	Solar	Traffic	Wikipedia
SN	0.060 $\pm$ 0.000	0.013 $\pm$ 0.000	0.512 $\pm$ 0.000	0.221 $\pm$ 0.000	0.423 $\pm$ 0.000
ARIMA	0.344 $\pm$ 0.000	0.013 $\pm$ 0.000	0.558 $\pm$ 0.003	0.486 $\pm$ 0.000	0.654 $\pm$ 0.000
ETS	0.056 $\pm$ 0.000	0.008 $\pm$ 0.000	0.550 $\pm$ 0.000	0.492 $\pm$ 0.000	0.651 $\pm$ 0.000
DeepAR	0.051 $\pm$ 0.000	0.013 $\pm$ 0.004	0.429 $\pm$ 0.015	0.103 $\pm$ 0.000	0.215 $\pm$ 0.003
WaveNet	0.058 $\pm$ 0.008	0.012 $\pm$ 0.001	0.360 $\pm$ 0.000	0.099 $\pm$ 0.000	0.207 $\pm$ 0.000
CSDI	0.051 $\pm$ 0.000	0.013 $\pm$ 0.000	0.360 $\pm$ 0.000	0.152 $\pm$ 0.000	0.318 $\pm$ 0.012
SSSD	0.048 $\pm$ 0.000	<u>0.010</u> $\pm$ 0.000	0.354 $\pm$ 0.024	0.107 $\pm$ 0.002	<u>0.209</u> $\pm$ 0.000
TSFlow	<u>0.045</u> $\pm$ 0.001	0.009 $\pm$ 0.001	<u>0.343</u> $\pm$ 0.002	0.083 $\pm$ 0.000	0.227 $\pm$ 0.000
FLOWTIME	0.042 $\pm$ 0.001	0.009 $\pm$ 0.001	0.284 $\pm$ 0.002	<u>0.089</u> $\pm$ 0.000	0.243 $\pm$ 0.002

Results Table 2 reports CRPS results for FLOWTIME and the baselines on five real-world time-series datasets. FLOWTIME obtains the best score on Electricity, Exchange, and Solar. On Electricity, its CRPS of 0.042 improves on the second-best model, TSFlow (0.045), by 6.7%. On Exchange, FLOWTIME matches TSFlow with CRPS 0.009 and remains ahead of the remaining statistical and neural baselines. On Solar, FLOWTIME obtains CRPS 0.284, a 17.2% reduction relative to TSFlow (0.343), the next-best method. On Traffic, FLOWTIME is second with CRPS 0.089, behind TSFlow (0.083) and ahead of the other listed methods. On Wikipedia, FLOWTIME obtains CRPS 0.243, trailing WaveNet (0.207) and SSSD (0.209). Overall, the results show that the autoregressive flow-matching parameterization is competitive across heterogeneous real-world forecasting tasks and achieves the strongest CRPS on three of the five benchmark datasets.

5 DISCUSSION

We introduced FLOWTIME, a probabilistic forecasting model that combines autoregressive temporal factorization with flow matching. The model represents the future trajectory distribution as a product of one-step conditional distributions and uses a shared conditional flow to model each next-observation law. This design couples the sequential inductive bias of autoregressive forecasting with the distributional flexibility of flow-based generative modeling. Experiments on stochastic dynamical systems and real-world datasets show that FLOWTIME is competitive with strong forecasting baselines. Relative to non-autoregressive flow matching, the autoregressive decomposition improves distributional coverage and uncertainty calibration in most dynamical-system settings, particularly in extrapolation regimes. These properties are important for deployment scenarios in which the evaluation distribution may differ from the training regime.

The method also has identifiable limitations. Autoregressive sampling is sequential at inference time, so it incurs higher latency than fully parallel non-autoregressive generation and may be less suitable for real-time forecasting constraints. In addition, the present implementation applies flow matching directly in observation space, which may be inefficient for high-dimensional or partially observed systems. Future work could develop latent-space variants, extend the formulation to irregularly sampled observations, and study applications in scientific modeling and decision-making

under uncertainty. Overall, FLOWTIME demonstrates that flow-based generative modeling can improve both predictive accuracy and uncertainty quantification when combined with an explicit autoregressive forecasting structure.

REFERENCES

- Michael S Albergo, Nicholas M Boffi, and Eric Vanden-Eijnden. Stochastic interpolants: A unifying framework for flows and diffusions. *arXiv preprint arXiv:2303.08797*, 2023.
- Juan Miguel Lopez Alcaraz and Nils Strodthoff. Diffusion-based time series imputation and forecasting with structured state space models. *arXiv preprint arXiv:2208.09399*, 2022.
- Alexander Alexandrov, Konstantinos Benidis, Michael Bohlke-Schneider, Valentin Flunkert, Jan Gasthaus, Tim Januschowski, Danielle C Maddix, Syama Rangapuram, David Salinas, Jasper Schulz, et al. GluonTS: Probabilistic and neural time series modeling in python. *Journal of Machine Learning Research*, 21(116):1–6, 2020.
- Ricky TQ Chen, Yulia Rubanova, Jesse Bettencourt, and David K Duvenaud. Neural ordinary differential equations. *Advances in Neural Information Processing Systems*, 31, 2018.
- Fatoumata Dama and Christine Sinoquet. Time series analysis and modeling to forecast: a survey. In *ArXiv:2104.00164*, 2021.
- Abhyuday Desai, Cynthia Freeman, Zuhui Wang, and Ian Beaver. TimeVAE: A variational auto-encoder for multivariate time series generation. *arXiv preprint arXiv:2111.08095*, 2021.
- Dheeru Dua, Casey Graff, et al. UCI machine learning repository. URL <http://archive.ics.uci.edu/ml>, 7(1):62, 2017.
- Patrick Esser, Sumith Kulal, Andreas Blattmann, Rahim Entezari, Jonas Müller, Harry Saini, Yam Levi, Dominik Lorenz, Axel Sauer, Frederic Boesel, et al. Scaling rectified flow transformers for high-resolution image synthesis. In *Forty-first International Conference on Machine Learning*, 2024.
- Richard FitzHugh. Impulses and physiological states in theoretical models of nerve membrane. *Biophysical Journal*, 1(6):445–466, 1961. doi: 10.1016/S0006-3495(61)86902-6.
- Jan Gasthaus, Konstantinos Benidis, Yuyang Wang, Syama Sundar Rangapuram, David Salinas, Valentin Flunkert, and Tim Januschowski. Probabilistic forecasting with spline quantile function RNNs. In *The 22nd International Conference on Artificial Intelligence and Statistics*, pp. 1901–1910. PMLR, 2019.
- Will Grathwohl, Ricky TQ Chen, Jesse Bettencourt, Ilya Sutskever, and David Duvenaud. FFIORD: Free-Form continuous dynamics for scalable reversible generative models. In *International Conference on Learning Representations*, 2018.
- Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. *Advances in Neural Information Processing Systems*, 33:6840–6851, 2020.
- Yang Hu, Xiao Wang, Lirong Wu, Huatian Zhang, Stan Z. Li, Sheng Wang, and Tianlong Chen. Fm-ts: Flow matching for time series generation. In *ArXiv:2411.07506v1 [cs.LG]*, 2024. URL <http://arxiv.org/abs/2411.07506>.
- Rob Hyndman, Anne B Koehler, J Keith Ord, and Ralph D Snyder. *Forecasting with Exponential Smoothing: The State Space Approach*. Springer Science & Business Media, 2008.
- Rob J Hyndman and Anne B Koehler. Another look at measures of forecast accuracy. *International Journal of Forecasting*, 22(4):679–688, 2006.
- Maximilian Karl, Maximilian Soelch, Justin Bayer, and Patrick Van der Smagt. Deep variational Bayes filters: Unsupervised learning of state space models from raw data. *arXiv preprint arXiv:1605.06432*, 2016.

432 Marcel Kollovich, Abdul Fatir Ansari, Michael Bohlke-Schneider, Jasper Zschiegner, Hao Wang,
433 and Yuyang Bernie Wang. Predict, refine, synthesize: Self-guiding diffusion models for proba-
434 bilistic time series forecasting. *Advances in Neural Information Processing Systems*, 36:28341–
435 28364, 2023.

436 Marcel Kollovich, Marten Lienen, David Lüdke, Leo Schwinn, and Stephan Günnemann. Flow
437 matching with gaussian process priors for probabilistic time series forecasting. In *The Thirteenth
438 International Conference on Learning Representations*, 2024.

439 Zhifeng Kong, Wei Ping, Jiaji Huang, Kexin Zhao, and Bryan Catanzaro. DiffWave: A versatile
440 diffusion model for audio synthesis. *arXiv preprint arXiv:2009.09761*, 2020.

441 Guokun Lai, Wei-Cheng Chang, Yiming Yang, and Hanxiao Liu. Modeling long- and short-term
442 temporal patterns with deep neural networks. In *The 41st International ACM SIGIR Conference
443 on Research & Development in Information Retrieval*, pp. 95–104, 2018.

444 R. Lefever and I. Prigogine. Symmetry breaking instabilities in dissipative systems. II. *The Journal
445 of Chemical Physics*, 48(4):1695–1700, 1968. doi: 10.1063/1.1668896.

446 Bryan Lim and Stefan Zohren. Time-series forecasting with deep learning: A survey. *Philosophical
447 Transactions of the Royal Society A*, 379(2194):20200209, 2021.

448 Yaron Lipman, Ricky TQ Chen, Heli Ben-Hamu, Maximilian Nickel, and Matthew Le. Flow match-
449 ing for generative modeling. In *The Eleventh International Conference on Learning Representa-
450 tions*, 2022.

451 Xingchao Liu, Chengyue Gong, et al. Flow straight and fast: Learning to generate and transfer data
452 with rectified flow. In *The Eleventh International Conference on Learning Representations*, 2022.

453 Edward Lorenz. Deterministic nonperiodic flow. *Journal of Atmospheric Sciences*, 20(2), 1963.

454 Alfred J. Lotka. *Elements of Physical Biology*. Williams & Wilkins, 1925.

455 Caspar Meijer and Lydia Y Chen. The rise of diffusion models in time-series forecasting. *arXiv
456 preprint arXiv:2401.03006*, 2024.

457 Adam Polyak, Amit Zohar, Andrew Brown, Andros Tjandra, Animesh Sinha, Ann Lee, Apoorv
458 Vyas, Bowen Shi, Chih-Yao Ma, Ching-Yao Chuang, et al. MovieGen: A cast of media founda-
459 tion models. *arXiv e-prints*, pp. arXiv–2410, 2024.

460 Kashif Rasul, Abdul-Saboor Sheikh, Ingmar Schuster, Urs Bergmann, and Roland Vollgraf. Mul-
461 tivariate probabilistic time series forecasting via conditioned normalizing flows. *arXiv preprint
462 arXiv:2002.06103*, 2020.

463 Kashif Rasul, Calvin Seward, Ingmar Schuster, and Roland Vollgraf. Autoregressive denoising dif-
464 fusion models for multivariate probabilistic time series forecasting. In *International Conference
465 on Machine Learning*, pp. 8857–8868. PMLR, 2021.

466 David Salinas, Michael Bohlke-Schneider, Laurent Callot, Roberto Medico, and Jan Gasthaus.
467 High-dimensional multivariate forecasting with low-rank gaussian copula processes. *Advances
468 in Neural Information Processing Systems*, 32, 2019.

469 David Salinas, Valentin Flunkert, Jan Gasthaus, and Tim Januschowski. DeepAR: Probabilistic
470 forecasting with autoregressive recurrent networks. *International Journal of Forecasting*, 36(3):
471 1181–1191, 2020.

472 Jascha Sohl-Dickstein, Eric Weiss, Niru Maheswaranathan, and Surya Ganguli. Deep unsupervised
473 learning using nonequilibrium thermodynamics. In *International Conference on Machine Learn-
474 ing*, pp. 2256–2265. PMLR, 2015.

475 Yang Song, Jascha Sohl-Dickstein, Diederik P Kingma, Abhishek Kumar, Stefano Ermon, and Ben
476 Poole. Score-based generative modeling through stochastic differential equations. *arXiv preprint
477 arXiv:2011.13456*, 2020.

486 Ella Tamir, Najwa Laabid, Markus Heinonen, Vikas Garg, and Arno Solin. Conditional flow match-
487 ing for time series modelling. In *ICML Workshop: Structured Probabilistic Inference and Gener-*
488 *ative Modeling*, 2024.

489 Yusuke Tashiro, Jiaming Song, Yang Song, and Stefano Ermon. CSDI: Conditional score-based
490 diffusion models for probabilistic time series imputation. *Advances in Neural Information Pro-*
491 *cessing Systems*, 34:24804–24816, 2021.

492 Alexander Tong, Kilian Fatras, Nikolay Malkin, Guillaume Huguet, Yanlei Zhang, Jarrid Rector-
493 Brooks, Guy Wolf, and Yoshua Bengio. Improving and generalizing flow-based generative models
494 with minibatch optimal transport. *arXiv preprint arXiv:2302.00482*, 2023.

495 Aaron Van Den Oord, Sander Dieleman, Heiga Zen, Karen Simonyan, Oriol Vinyals, Alex Graves,
496 Nal Kalchbrenner, Andrew Senior, Koray Kavukcuoglu, et al. WaveNet: A generative model for
497 raw audio. *arXiv preprint arXiv:1609.03499*, 12, 2016.

498 B. van der Pol. On ‘relaxation-oscillations’. *The London, Edinburgh, and Dublin Philosophical*
499 *Magazine and Journal of Science*, 2(11):978–992, 1926. doi: 10.1080/14786442608564127.

500 Ronald J Williams and David Zipser. A learning algorithm for continually running fully recurrent
501 neural networks. *Neural Computation*, 1(2):270–280, 1989.

502 Robert L Winkler, Javier Munoz, José L Cervera, José M Bernardo, Gail Blattenberger, Joseph B
503 Kadane, Dennis V Lindley, Allan H Murphy, Robert M Oliver, and David Ríos-Insua. Scoring
504 rules and the evaluation of probabilities. *Test*, 5:1–60, 1996.

505 Jiexia Ye, Weiqi Zhang, Ke Yi, Yongzi Yu, Ziyue Li, Jia Li, and Fugee Tsung. A survey of time
506 series foundation models: Generalizing time series representation with large language model. In
507 *ArXiv:2104.00164*, 2024.

508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539

A DATASETS

A.1 DYNAMICAL SYSTEMS

Training and test trajectories were generated for five classical stochastic dynamical systems: Lorenz, FitzHugh-Nagumo, Lotka-Volterra, Brusselator, and Van der Pol. For each system, data were produced by numerical integration of the corresponding stochastic differential equation (SDE). Each SDE has the form

$$d\mathbf{x}(t) = \mathbf{f}(\mathbf{x}(t), t; \Theta) dt + \sigma d\mathbf{W}(t),$$

where f denotes the deterministic drift, Θ denotes the system parameters, σ is a constant diffusion term, and $\mathbf{W}(t)$ is standard Brownian motion. For each system, 2400 initial conditions were sampled uniformly from the range specified in Table 3. One trajectory was generated per initial condition with the Euler-Heun method using a fixed step size, and 200 equally spaced time points were retained. Within each trajectory, the first 75 time points form the observed context, the next 75 define the prediction window, and the final 50 define the extrapolation window. Of the 2400 generated trajectories, 2000 were assigned to training and 400 to testing.

Table 3: Specification of the stochastic dynamical systems used for simulation. $\mathbf{f}(\mathbf{x})$ denotes the drift function, Θ denotes system parameters, σ denotes the constant diffusion term, and $p(\mathbf{x}(0))$ denotes the initial-condition distribution. Each system is integrated over $[t_0, t_1]$ with 200 Euler-Heun steps.

System	$f(\mathbf{x})$	Θ	Σ	$p(\mathbf{x}(0))$	$[t_0, t_1]$
Lorenz	$f_1 = \sigma(x_2 - x_1)$	$\sigma = 10$	(1.5, 1.5, 1.5)	$\mathcal{U}([0, 10])$	[0, 2]
	$f_2 = x_1(\rho - x_3) - x_2$	$\rho = 28$			
	$f_3 = x_1 x_2 - \beta x_3$	$\beta = \frac{8}{3}$			
FitzHugh-Nagumo	$f_1 = x_1 - \frac{x_1^3}{3} - x_2 + I$	$a = 0.7$	(1.5, 1.5)	$\mathcal{U}([-2, 2])$	[0, 10]
	$f_2 = \frac{x_1 + a - b x_2}{\tau}$	$b = 0.8$			
		$\tau = 12.5$			
Lotka-Volterra		$I = 0.5$			
	$f_1 = \alpha x_1 - \beta x_1 x_2$	$\alpha = 1.3$	(1.5, 1.5)	$\mathcal{U}([0, 5])$	[0, 20]
	$f_2 = -\delta x_2 + \gamma x_1 x_2$	$\beta = 0.9$			
		$\gamma = 0.8$			
Brusselator		$\delta = 1.8$	(1.5, 1.5)	$\mathcal{U}([0, 2])$	[0, 20]
	$f_1 = A + x_1^2 x_2 - (B + 1) x_1$	$A = 1.0$			
	$f_2 = B x_1 - x_1^2 x_2$	$B = 3.0$			
Van der Pol	$f_1 = x_2$	$\mu = 0.1$	(1.5, 1.5)	$\mathcal{U}([-2, 2])$	[0, 20]
	$f_2 = \mu(1 - x_1^2) x_2 - x_1$				

A.2 REAL-WORLD DATASETS

Model performance is evaluated on five GluonTS datasets (Alexandrov et al., 2020). We use the preprocessed versions described by (Salinas et al., 2019); their dimensionality, domain, sampling frequency, available training length, and prediction horizon are summarized in Table 4.

Table 4: Dataset specifications used in the real-world experiments, including dimension n , state domain Ω , sampling frequency, number of training timesteps, and prediction length.

Dataset	n	Ω	freq.	timesteps	prediction length
Electricity	370	$\mathbb{R}^+$	hourly	5833	24
Exchange	8	$\mathbb{R}^+$	daily	6071	30
Solar	137	$\mathbb{R}^+$	hourly	7009	24
Traffic	963	(0, 1)	hourly	4001	24
Wikipedia	2000	$\mathbb{N}$	daily	792	30

B NON-AUTOREGRESSIVE FLOW MATCHING BASELINE

Problem Setting The non-autoregressive baseline uses the same forecasting inputs as Section 3: past observations $\mathbf{Y}_l$ and covariates $\mathbf{C}$. Its target remains the conditional distribution $p(\mathbf{Y}_f|\mathbf{Y}_l, \mathbf{C})$. In contrast to FLOWTIME, however, this baseline models the entire future trajectory $\mathbf{Y}_f \in \mathbb{R}^{f \times n}$ as a single random object, without temporal factorization:

$$p(\mathbf{Y}_f|\mathbf{Y}_l, \mathbf{C}) = p(\mathbf{y}_{\tau+1}, \dots, \mathbf{y}_{\tau+f}|\mathbf{Y}_l, \mathbf{C}) \quad (9)$$

This parameterization can represent temporal correlations across all future time steps, but it requires fitting a higher-dimensional conditional distribution.

Training The baseline constructs a trajectory-level probability path $(p^s(\mathbf{Y}_f))^{0 \leq s \leq 1}$ that transports samples from the prior $p^0(\mathbf{Y}_f) = \mathcal{N}(0, \Sigma)$ to the target distribution $p^1(\mathbf{Y}_f) \approx p(\mathbf{Y}_f|\mathbf{Y}_l, \mathbf{C})$. Here, Σ is block diagonal. The prior corresponds to independent Brownian-motion processes along each dimension, with Brownian motion represented as a stochastic process $W: [\tau + 1, \tau + f] \rightarrow \mathbb{R}^n$.

The conditional probability path is defined as a linear interpolation bridge with Brownian noise:

$$p^s(\mathbf{Y}_f|\mathbf{z}) = \mathcal{N}((1-s)\mathbf{Y}_f^0 + s\mathbf{Y}_f^1, \sigma^2 s(1-s)\Sigma) \quad (10)$$

where $\mathbf{z} = (\mathbf{Y}_f^0, \mathbf{Y}_f^1)$, with $\mathbf{Y}_f^0 \sim p^0$ and $\mathbf{Y}_f^1 \sim p^1$. The associated conditional velocity field is:

$$\mu(\mathbf{Y}_f, s|\mathbf{z}) = \mathbf{Y}_f^1 - \mathbf{Y}_f^0 + \frac{\sigma^2(1-2s)}{2}\Sigma^{-1}(\mathbf{Y}_f - ((1-s)\mathbf{Y}_f^0 + s\mathbf{Y}_f^1)) \quad (11)$$

In the limit $\sigma^2 \rightarrow 0$, this expression reduces to the straight-path velocity $\mu(\mathbf{Y}_f, s|\mathbf{z}) = \mathbf{Y}_f^1 - \mathbf{Y}_f^0$. The baseline learns a neural velocity field ν_θ whose input is the complete future trajectory. The context encoder is analogous to Eq. (7):

$$\mathbf{h} = \zeta_\phi(\mathbf{Y}_l, \mathbf{C}_{\tau-l:\tau+f}) \quad (12)$$

where ζ_ϕ receives both past and future covariates. The corresponding training objective is:

$$\mathcal{L}(\theta, \phi) = \mathbb{E}_{\mathbf{z} \sim \pi_{0,1}, s \sim \mathcal{U}(0,1), \mathbf{Y}_f^s \sim p^s(\mathbf{Y}_f|\mathbf{z})} \|\mu(\mathbf{Y}_f^s, s|\mathbf{z}) - \nu_\theta(\mathbf{Y}_f^s, \mathbf{h}, \mathbf{C}_{\tau+1:\tau+f}, s)\|^2 \quad (13)$$

where ν_θ is implemented as a sequential neural network, such as a Transformer or RNN, over the complete trajectory. Sampling is performed by solving the following trajectory-level ODE:

$$\frac{d\psi(\mathbf{Y}_f, s)}{ds} = \nu_\theta(\psi(\mathbf{Y}_f, s), \mathbf{h}, \mathbf{C}_{\tau+1:\tau+f}, s), \quad \psi(\mathbf{Y}_f, 0) \sim p^0 \quad (14)$$

with a numerical ODE solver. The resulting terminal states are joint samples from $p(\mathbf{Y}_f|\mathbf{Y}_l, \mathbf{C})$ generated without autoregressive decomposition.

Implementation Details This baseline is used for the dynamical-system forecasting experiments. In this configuration, ν_θ is a 4-layer bidirectional LSTM with 128 hidden units. All other training hyperparameters match those in Section 4.1. Training to convergence required approximately ~ 2 minutes on one A100-SXM4-40GB NVIDIA GPU.

Key Differences from Autoregressive Variant The non-autoregressive baseline differs from FLOWTIME in its statistical parameterization and computational profile. First, FLOWTIME factorizes the future trajectory distribution through a Markovian product over time steps, whereas the non-autoregressive baseline treats $\mathbf{Y}_f$ as a single high-dimensional random variable. This preserves cross-temporal dependencies directly, but it requires learning a distribution over $\mathbb{R}^{n \times f}$. Second, the non-autoregressive model processes entire trajectories with sequential velocity networks, enabling parallel generation of all future time points after ODE integration while increasing memory requirements. Third, the two models represent uncertainty propagation differently: the autoregressive model propagates uncertainty through the chain of one-step conditional distributions, whereas the non-autoregressive model represents joint trajectory uncertainty through whole-window samples without an explicit mechanism for modeling error accumulation. These differences induce complementary trade-offs: the autoregressive variant provides explicit uncertainty propagation and window-based computation under a finite-order Markov assumption, while the non-autoregressive baseline preserves full-window temporal dependence at higher computational and memory cost.

Algorithm 3 Non-Autoregressive Flow Matching: Training

Require: Dataset $\mathcal{D}$, networks ν_θ, ζ_ϕ

- 1: **while** not converged **do**
- 2: Sample $\mathbf{Y}_f^0 \sim \mathcal{N}(0, \Sigma)$ and $(\mathbf{Y}_f^1, \mathbf{Y}_l, \mathbf{C}) \sim \mathcal{D}$
- 3: Set $\mathbf{h} = \zeta_\phi(\mathbf{Y}_l, \mathbf{C}_{\tau-l:\tau+f})$
- 4: Sample $s \sim \mathcal{U}(0, 1)$
- 5: $\mathbf{Y}_f^s \sim \mathcal{N}((1-s)\mathbf{Y}_f^0 + s\mathbf{Y}_f^1, \sigma^2 s(1-s)\Sigma)$
- 6: Compute target $\mu(\mathbf{Y}_f^s, s | \mathbf{z})$
- 7: Update θ, ϕ to minimize $\|\mu(\cdot) - \nu_\theta(\mathbf{Y}_f^s, \mathbf{h}, \mathbf{C}_{\tau+1:\tau+f}, s)\|_{\Sigma^{-1}}^2$
- 8: **end while**

Algorithm 4 Non-Autoregressive Flow Matching: Sampling

Require: $\mathbf{Y}_l, \mathbf{C}$, networks ν_θ, ζ_ϕ

- 1: Encode context: $\mathbf{h} = \zeta_\phi(\mathbf{Y}_l, \mathbf{C})$
- 2: Sample initial trajectory $\mathbf{Y}_f^0 \sim \mathcal{N}(0, \Sigma)$
- 3: Solve $\frac{d\psi(\mathbf{Y}_f, s)}{ds} = \nu_\theta(\psi(\mathbf{Y}_f, s), \mathbf{h}, \mathbf{C}_{\tau+1:\tau+f}, s)$
- 4: Return $\psi(\mathbf{Y}_f, 1)$

C EVALUATION METRICS

This section defines the evaluation metrics used to quantify forecast quality. For an n -dimensional future trajectory $\mathbf{Y}_f = \{\mathbf{y}_{\tau+1}, \dots, \mathbf{y}_{\tau+f}\}$ and either its predictive distribution or Monte Carlo samples from the probabilistic model, we report the Continuous Ranked Probability Score (CRPS) and the Normalized Root Mean Squared Error (NRMSE).

C.1 CONTINUOUS RANKED PROBABILITY SCORE (CRPS)

CRPS is a proper scoring rule for evaluating probabilistic forecasts. For a univariate random variable X with cumulative distribution function (CDF) F_X and realized value x , CRPS is defined as:

$$\text{CRPS}(F_X, x) = \int_{-\infty}^{+\infty} (F_X(y) - \mathbf{1}\{y \geq x\})^2 dy, \quad (15)$$

where $\mathbf{1}\{\cdot\}$ denotes the indicator function. CRPS evaluates the discrepancy between the full predictive CDF and the empirical step function induced by the observation x . Lower values correspond to predictive distributions with better calibration and sharper accuracy.

For a forecasting horizon, CRPS is computed at each future time step and then averaged over the horizon:

$$\text{CRPS}_{\text{avg}} = \frac{1}{f} \sum_{k=1}^f \text{CRPS}(F_{X_{\tau+k}}, y_{\tau+k}), \quad (16)$$

where $F_{X_{\tau+k}}$ denotes the predicted CDF, or the empirical CDF estimated from samples, at horizon index k , and $y_{\tau+k}$ is the corresponding ground-truth observation.

C.2 NORMALIZED ROOT MEAN SQUARED ERROR (NRMSE)

Root Mean Squared Error (RMSE) measures the accuracy of point forecasts. For predictions $\{\hat{y}_{\tau+1}, \dots, \hat{y}_{\tau+f}\}$ and ground-truth values $\{y_{\tau+1}, \dots, y_{\tau+f}\}$, RMSE is defined as:

$$\text{RMSE} = \sqrt{\frac{1}{f} \sum_{k=1}^f (\hat{y}_{\tau+k} - y_{\tau+k})^2}. \quad (17)$$

To make RMSE scale invariant and comparable across datasets with different magnitudes, we report Normalized RMSE (NRMSE). The normalization used here divides by the standard deviation of the ground-truth series, σ_y :

$$\text{NRMSE} = \frac{\text{RMSE}}{\sigma_y}, \quad (18)$$

where σ_y is the sample standard deviation of the observations. Other normalizations, such as the empirical range ($\max(y) - \min(y)$), may be used in different applications. Lower NRMSE indicates smaller prediction error relative to the variability of the evaluated time series.